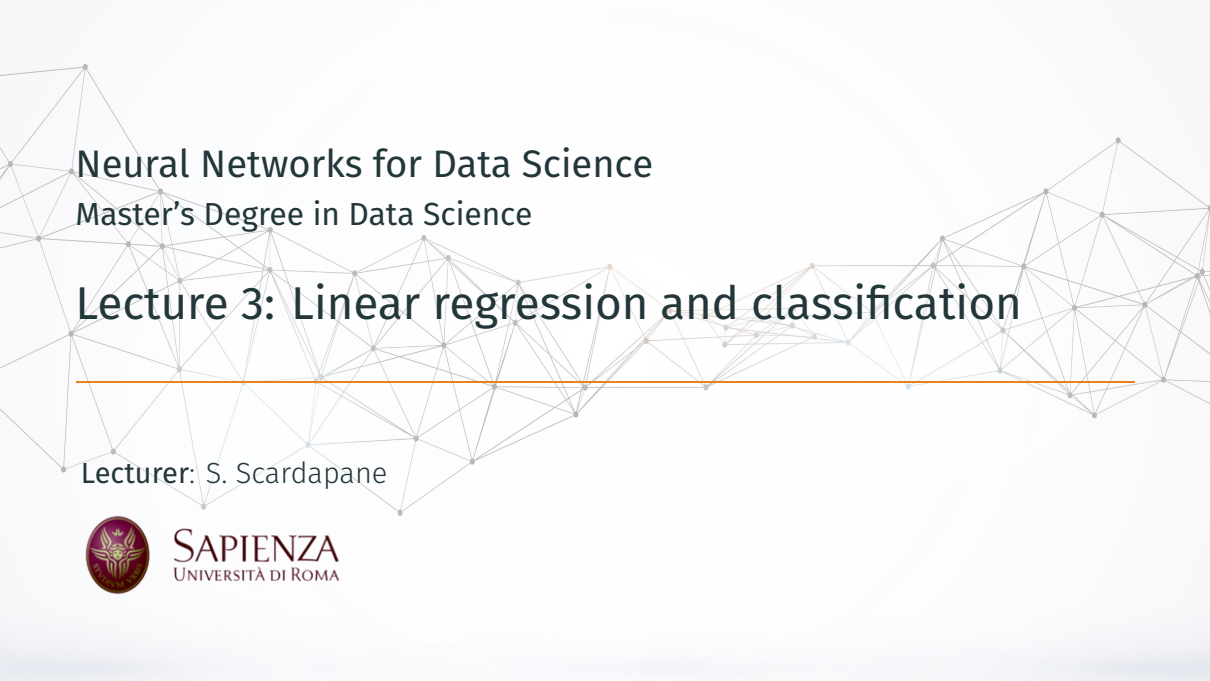




Neural Networks for Data Science

Master's Degree in Data Science

Lecture 3: Linear regression and classification

Lecturer: S. Scardapane



SAPIENZA
UNIVERSITÀ DI ROMA

Supervised learning

Setup and data splits

A supervised dataset is a set of N input-target pairs:

$$\mathcal{S} = \{(x_i, y_i)\}_{i=1}^N . \quad (1)$$

A model f_{θ} maps an input x to a prediction $\hat{y}$:

$$\hat{y} = f_{\theta}(x) . \quad (2)$$

We fit θ on $\mathcal{S}$ and evaluate the model on examples that were not used for training.

We split the available examples into three disjoint sets:

$$\mathcal{S} = \mathcal{S}_{\text{train}} \dot{\cup} \mathcal{S}_{\text{val}} \dot{\cup} \mathcal{S}_{\text{test}} . \quad (3)$$

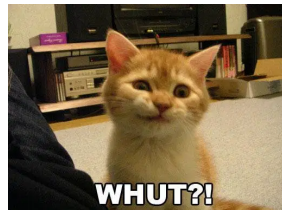
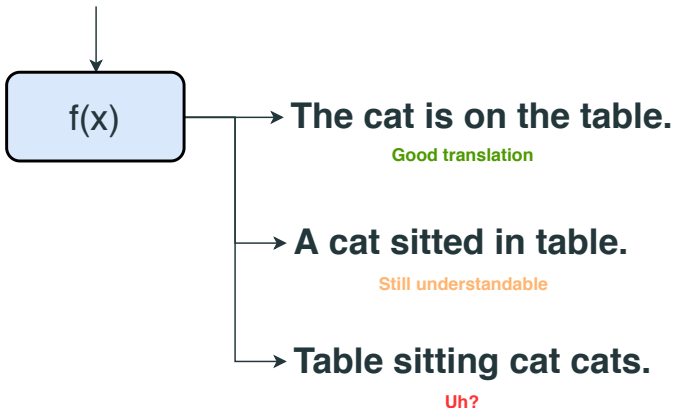
- ▶ $\mathcal{S}_{\text{train}}$ is used to fit the parameters θ .
- ▶ $\mathcal{S}_{\text{val}}$ is used to select the model and its hyper-parameters.
- ▶ $\mathcal{S}_{\text{test}}$ is used for the final evaluation.

We usually assume independent samples from the same unknown distribution $p(x, y)$ (*identically* and *independently* distributed, i.i.d.). If the deployment distribution differs, we have a **distribution shift**.

Supervised learning

Loss functions

"Il gatto è sul tavolo."



A **loss function** assigns a scalar cost to a prediction–target pair:

$$\ell(\hat{y}, y) \in \mathbb{R}, \quad \hat{y} = f_{\theta}(x). \quad (4)$$

A smaller loss means a better prediction. Learning amounts to choosing θ so that predictions have low loss on the training data.

For gradient-based learning, we usually choose losses that are **differentiable** with respect to the model parameters.

Ideally, we would minimize the loss on the entire data distribution:

$$\mathcal{R}(\boldsymbol{\theta}) = \mathbb{E}_{p(x,y)} [\ell(f_{\boldsymbol{\theta}}(x), y)] . \quad (5)$$

This is the **expected risk**, but $p(x, y)$ is unknown. We replace the expectation with the average over the training data:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(x_i), y_i) , \quad \boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) . \quad (6)$$

A small empirical risk does not guarantee a small expected risk. Their difference is the **generalization gap**.

Consider a model that memorizes the training set:

$$f(x) = \begin{cases} y_i & \text{if } x = x_i \text{ for some } (x_i, y_i) \in \mathcal{S}_{\text{train}}, \\ \bar{y} & \text{otherwise.} \end{cases} \quad (7)$$

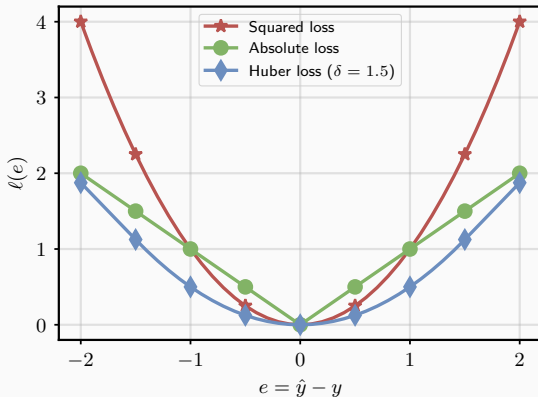
It predicts every training target exactly. For any new input, however, it returns the same default value $\bar{y}$.

$$\mathcal{L}(\theta) = 0 \quad \not\Rightarrow \quad \mathcal{R}(\theta) \text{ is small.}$$

This is **overfitting**: fitting the training data without learning a rule that generalizes.

For **regression** we have $y, \hat{y} \in \mathbb{R}$, and we can write the prediction error as $e = \hat{y} - y$.

- **Squared loss:** $\ell(e) = e^2$. Smooth, but gives large errors more weight.
- **Absolute loss:** $\ell(e) = |e|$. Less sensitive to outliers, but not differentiable at $e = 0$.
- **Huber loss:** quadratic near zero and linear for large $|e|$.



Maximum likelihood gives squared loss under Gaussian noise and absolute loss under Laplace noise, see the book.

Linear models

Linear models for regression

For an input $\mathbf{x} \in \mathbb{R}^D$, a scalar linear model is defined as:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^D w_j x_j + b. \quad (8)$$

The parameters are $\mathbf{w} \in \mathbb{R}^D$ and $b \in \mathbb{R}$, with $\boldsymbol{\theta} = \{\mathbf{w}, b\}$. The scalar b is the **bias**; we will sometimes omit it to simplify the notation. We call each component x_j a **feature**, and datasets of this form **tabular**.

Combining the squared loss with a linear model results in the **least-squares** optimization problem:

$$\text{LS}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2. \quad (9)$$

We can vectorize LS as:

$$\text{LS}(\mathbf{w}) = \frac{1}{n} \left\| \underset{(n)}{\mathbf{y}} - \underset{(n,d)}{\mathbf{X}} \underset{(d)}{\mathbf{w}} \right\|^2, \quad (10)$$

where $[\mathbf{X}]_i = \mathbf{x}_i$ and $[\mathbf{y}]_i = y_i$.

Linear models

Solving the LS problem

LS is a convex problem, with a simple gradient (**normal equations**):

$$\nabla \text{LS}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}). \quad (11)$$

LS is special in the sense that $\nabla \text{LS}(\mathbf{w}) = 0$ is a linear equation that can be solved explicitly:

$$\mathbf{w}^* = \underset{(d,d)}{(\mathbf{X}^\top \mathbf{X})^{-1}} \underset{(d,n)(n)}{\mathbf{X}^\top} \underset{(n)}{\mathbf{y}} = \underset{(d,n)}{\mathbf{X}^\top} \underset{(n,n)}{(\mathbf{X}\mathbf{X}^\top)^{-1}} \underset{(n)}{\mathbf{y}}. \quad (12)$$

The terms $\underset{(d,d)}{(\mathbf{X}^\top \mathbf{X})^{-1}} \underset{(d,n)(n)}{\mathbf{X}^\top}$ and $\underset{(d,n)}{\mathbf{X}^\top} \underset{(n,n)}{(\mathbf{X}\mathbf{X}^\top)^{-1}}$ are called the **pseudoinverses** of $\mathbf{X}$, and they require the corresponding matrices to be invertible (full rank).

Numerical problems in the inversion of $(\mathbf{X}^\top \mathbf{X})$ can be solved by adding a small amount of ℓ_2 **regularization** (**ridge regression**):

$$\text{LS-REG}(\mathbf{w}) = \text{LS}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2, \quad (13)$$

for some $\lambda > 0$. This makes the problem *strictly* convex and forces the solution to be contained in a ball of given radius, modifying the gradient and the explicit solution as:

$$\nabla \text{LS-REG}(\mathbf{w}) = \nabla \text{LS}(\mathbf{w}) + \lambda \mathbf{w}. \quad (14)$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (15)$$

where $\mathbf{I}$ is the identity matrix of appropriate shape.

Generating some data:

```
1 # Linear model with unknown coefficients
2 X = tf.random.normal((10, 5))
3 y = X @ tf.random.normal((5, 1))
```

Computing a linear model:

```
1 w = tf.random.normal((5, 1))
2 yhat = X @ w # (10, 1)
```

Computing the objective function:

```
1 mse = tf.reduce_mean((y - yhat)**2)
```

Explicit solution (numerically unstable):

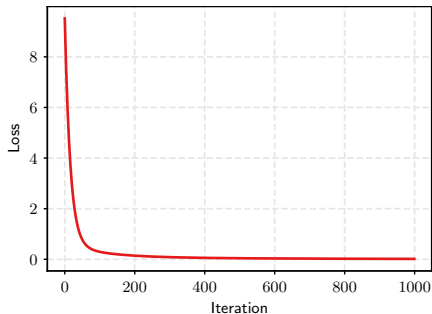
```
1 wopt = tf.linalg.inv(tf.transpose(X) @ X) @ tf.transpose(X) @ y
```

Explicit solution (better numerical conditioning):

```
1 wopt = tf.linalg.solve(tf.transpose(X) @ X, tf.transpose(X) @ y)
```

Simple implementation of gradient descent:

```
1 for i in range(15000):  
2     # Note the sign: the derivative has a minus!  
3     w = w + 0.001 * tf.transpose(X) @ (y - X @ w)
```



Matrix multiplication $\mathbf{A} \mathbf{B}$ has complexity $\mathcal{O}(abc)$, while matrix inversion has complexity (roughly) cubic.

For large models and datasets, we want algorithms that can scale linearly in both n and d : we will see that gradient descent can satisfy this property.

Linear models

Linear models for classification

In **classification**, y is an integer $\{0, \dots, c - 1\}$, such that $y_i = j$ means that $\mathbf{x}_i$ is of class j .

For example, with $c = 3$ we might have:

- ▶ $y = 0$: the email is spam;
- ▶ $y = 1$: the email is legit;
- ▶ $y = 2$: the email is dubious.

Solving these as regression tasks is generally not an optimal choice: among other things, it is not guaranteed that classes have a definite ordering.

A common solution is to predict a **probability distribution** over the classes.

A vector $\mathbf{a}$ belongs to the **probability simplex** Δ_c if:
(c)

$$\sum_i [\mathbf{a}]_i = 1, \quad [\mathbf{a}]_i \geq 0. \quad (16)$$

If $f(\mathbf{x}) = \hat{\mathbf{y}} \in \Delta_c$, we can interpret it as a **categorical probability distribution**, e.g., we can select the class with highest probability as:

$$\text{class} = \arg \max_i [\hat{\mathbf{y}}]_i. \quad (17)$$

Note that we *cannot* easily predict an integer with our models, because it would require some form of threshold operation which is not compatible with gradient descent (gradient zero almost everywhere).

Predicting a probability distribution can be seen as a *soft* approximation to this problem.

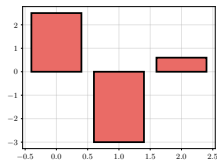
The **softmax** function maps any vector to the probability simplex:

$$[\text{softmax}(\mathbf{a})]_i = \frac{\exp(a_i)}{\sum_j \exp(a_j)} \quad (18)$$

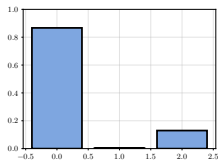
The numerator ensures that all outputs are positive, while the denominator ensures that the final vector sums to 1. It can be seen as a soft approximation to the argmax (a better name is in fact *softargmax*).

If desired, we can control the approximation with an additional hyper-parameter τ called **temperature**:

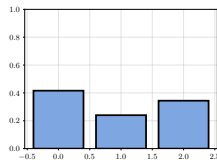
$$[\text{softmax}(\mathbf{a})]_i = \frac{\exp(a_i/\tau)}{\sum_j \exp(a_j/\tau)} \quad (19)$$



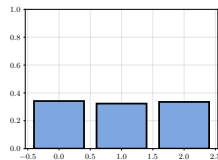
(a) Logits



(b) $\tau = 1$



(c) $\tau = 10$



(d) $\tau = 100$

Figure 1: Input logits (red) and softmax outputs with three different temperature values.

Our linear model for classification becomes:

$$f(\mathbf{x}) = \underset{(c)}{\text{softmax}}\left(\underset{(c,d)}{\mathbf{W}} \underset{(d)}{\mathbf{x}}\right) \quad (20)$$

The pre-softmax values $\underset{(c)}{\mathbf{W}}\mathbf{x}$ are called the **logits** of the model. By explicitly writing the biases $\mathbf{b}$:

$$f(\mathbf{x}) = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b}). \quad (21)$$

In order to compare the predictions with the ground truth, we encode our targets using a **one-hot encoding**. Given a pair $(\mathbf{x}, \mathbf{y})$:

$$y_i = \begin{cases} 1 & \text{if } \mathbf{x} \text{ is of class } i, \\ 0 & \text{otherwise.} \end{cases} \quad (22)$$

For example, with 3 classes $\{\text{cat}, \text{dog}, \text{other}\}$:

$$\text{cat} = [1, 0, 0] \quad \text{dog} = [0, 1, 0] \quad \text{other} = [0, 0, 1]. \quad (23)$$

This is a probability distribution putting all the **mass** on a single class.

Finally, we need a loss function l to compare two probability distributions.

The **cross-entropy** loss is defined for two vectors $\mathbf{y}, \hat{\mathbf{y}} \in \Delta_c$ as:

$$\text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \log(\hat{y}_i) . \quad (24)$$

The CE loss can be derived from the maximum likelihood principle under the assumption that $f(\mathbf{x})$ encodes a **categorical** distribution (try it!).¹

¹The equation for the categorical distribution in this case would be $p(\mathbf{y}|f(\mathbf{x})) = \prod_i [f(\mathbf{x})]_i^{y_i}$.

The CE loss is generic for *any* pair of vectors belonging to the probability simplex. However, in our case $\mathbf{y}$ is always a one-hot encoded vector.

Denote by t the index of the class, $t = \arg \max \mathbf{y}$, the CE loss simplifies since only one term in the summation will be different from 0:

$$\text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = -\log(\hat{y}_t). \quad (25)$$

Hence, CE can be interpreted as maximizing the probability of the true class at the expense of all other outputs.

A **logistic regression** is a linear model $f(\mathbf{x}) = \text{softmax}(\mathbf{W}\mathbf{x})$ trained by optimizing the cross-entropy:

$$\text{LR}(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \text{CE}(\mathbf{y}_i, f(\mathbf{x}_i)) . \quad (26)$$

It is not possible to solve the logistic regression problem explicitly. A linear model for classification has dc parameters (or $dc + c$ with biases).

A special case is **binary classification**, where $c = 2$. In this case, we can predict a single scalar value $f(\mathbf{x}) \in [0, 1]$ since:

$$f(\mathbf{x}) \quad \text{probability of class 1 ,} \quad (27)$$

$$1 - f(\mathbf{x}) \quad \text{probability of class 2 .} \quad (28)$$

The softmax function simplifies to the **sigmoid** function:

The sigmoid $\sigma(s) \in [0, 1]$ is defined as:

$$\sigma(s) = \frac{1}{1 + \exp(-s)} . \quad (29)$$

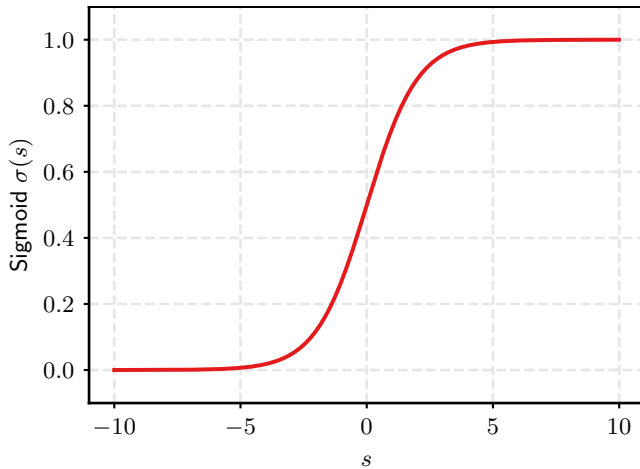


Figure 2: A visualization of the sigmoid function. Note that 0 and 1 are only approached asymptotically.

Combining everything, we obtain a binary version of the logistic regression algorithm:

$$\text{BIN-LR}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \left[\underbrace{-y_i \log(\sigma(\mathbf{w}^\top \mathbf{x}))}_{\text{Class 1}} - \underbrace{(1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}))}_{\text{Class 2}} \right] \quad (30)$$

In this case, we can obtain the most probable class from the model as:

$$\text{class} = \begin{cases} 1 & \text{if } \sigma(\mathbf{w}^\top \mathbf{x}) > 0.5, \\ 0 & \text{otherwise .} \end{cases} \quad (31)$$

By manually differentiating we obtain:

$$\sigma'(s) = \sigma(s)(1 - \sigma(s)). \quad (32)$$

Plugging this into the gradient computation we obtain:

$$\nabla \text{BIN-LR}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i) \mathbf{x}_i, \quad (33)$$

showing its similarity to the regression case.

```
1 from tensorflow.keras.metrics import *  
2  
3 # The one we have described up to now.  
4 categorical_crossentropy(ytrue, yhat)  
5  
6 # ytrue should contain the indexes of the classes instead of the  
7 # one-hot encodings.  
8 sparse_categorical_crossentropy(ytrue, yhat)  
9  
10 # Numerically-stable versions requiring the logits as inputs  
11 categorical_crossentropy(ytrue, yhat, from_logits=True)  
12 sparse_categorical_crossentropy(ytrue, yhat, from_logits=True)
```

Why a variant with logits in input? Note that the i th term of the cross-entropy wrt the logits $\mathbf{p}$ is given by:

$$-\log \left(\frac{\exp p_i}{\sum_j \exp p_j} \right) . \quad (34)$$

In practice, this can be *highly* unstable. However, it can be rewritten as:

$$-p_i + \text{logsumexp}(\mathbf{p}) , \quad (35)$$

where logsumexp is defined as $\text{logsumexp}(\mathbf{p}) = \log(\sum_i \exp p_i)$.

The reason this is important is that the logsumexp function is invariant in the following sense:

$$\text{logsumexp}(\mathbf{p}) = \log \left(\sum_i \exp(p_i - c) \right) + c, \quad (36)$$

where c is an arbitrary constant. By setting $c = \max(\mathbf{p})$, we can ensure numerical instabilities never occur.

In this sense, softmax can be interpreted as part of the model *or* as part of the loss; this is not an issue, since $\arg \max$ only cares about the relative ranking of the values, which is not changed by the softmax.

Linear models

Calibration and a probabilistic
formulation

A common misconception when doing classification is that $[f(x)]_i$ can be immediately interpreted as *the probability of pattern x being of class i* .

However, this is only true whenever the trained model satisfies:

$$p(y = i | x) = [f(x)]_i. \quad (37)$$

We say the model is well **calibrated**, but this must be checked manually.

Guo, C., et al.. On calibration of modern neural networks. ICML 2017.

To measure calibration, we keep a separate validation set, and we split the interval $[0, 1]$ into m equispaced bins (each of size $1/m$). Define:

- ▶ B_m the number of samples from the validation set, whose predicted confidence falls in bin m .
- ▶ p_m the average confidence of the network for that bin.
- ▶ a_m the average accuracy of the network for these elements.

Then, the **expected calibration error** (ECE) is given by:

$$\text{ECE} = \sum_i \frac{|B_i|}{n} |a_i - p_i| \quad (38)$$

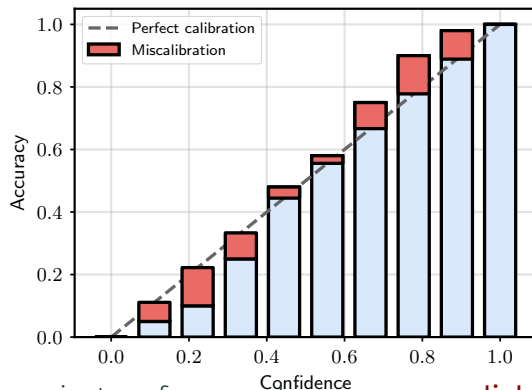


Figure 3: Plotting a_m against p_m for every bin gives us a **reliability plot** (from Guo et al., 2017).

This topic is important because more complex networks may be highly over (or under) confident, with many methods to improve it (temperature scaling, logit normalization, ...).

A simple (and popular) option is to decrease the weight given to ‘easy’ samples using a variant of cross-entropy call the **focal loss**:

$$\text{FL}_\alpha(\mathbf{y}, \hat{\mathbf{y}}) = -(1 - \hat{y}_c)^\alpha \log \hat{y}_c, \quad (39)$$

where $c = \arg \max \mathbf{y}$.

Mukhoti, J., et al., 2020. **Calibrating deep neural networks using focal loss**. *Advances in Neural Information Processing Systems*, 33, pp. 15288-15299.

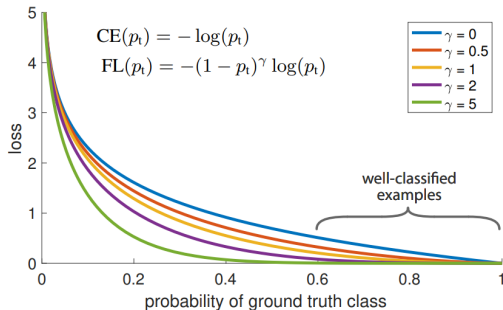


Figure 1. We propose a novel loss we term the *Focal Loss* that adds a factor $(1 - p_t)^\gamma$ to the standard cross entropy criterion. Setting $\gamma > 0$ reduces the relative loss for well-classified examples ($p_t > .5$), putting more focus on hard, misclassified examples. As

- ▶ **Chapters 3 and 4** of the book, along with the suggested exercises in Chapter 4.